



[BIT703 Web Technologies](#) / [Contents](#) / [Topic 12: Web application performance](#) / [Testing techniques](#)

# Black-box techniques 15m



© © Getty Images



[Skip to main content](#)

Black-box testing techniques are used to evaluate the functionality of the software system or application under test (AUT) based on business requirements and software specifications (specification-based techniques). No knowledge of the internal code structure is required or known (hence the name 'black box'); you are testing from the end-user perspective. The black box is any software system you want to test, such as an operating system, website or your own application or work strand.

Black-box test cases are designed from the requirement specification documents and follows more or less the same path as that described for functional testing:

1. Identify and provide appropriate test input data
2. Compute the expected outcomes with the test input values
3. Execute test cases
4. Compare the actual and computed expected results
5. Log defects, fix defects and retest.

Functional, nonfunctional, regression and integration testing are all types of black-box testing.

Different tools can be used for black-box testing, such as:

- Selenium for functional testing
- OpenText Professional Performance Engineering (LoadRunner Professional) or JMeter for nonfunctional testing.

Different testing techniques are used to design test cases for black-box testing. Have a look at the following black-box testing techniques.

## Equivalence class partitioning

This is a technique to divide the input data into different classes based on similarity. It helps avoid testing too many test cases while maintaining reasonable test coverage. This helps to reduce the range of required tests as you are not expected to test for every possible scenario.

For example, assume you have an AUT that accepts integers in the range of 0–9.

[Skip to main content](#)

A valid equivalence class partition is 0–9 inclusive.

An invalid equivalence class partition is integers <0, >9, decimal numbers, letters and non-numeric characters.

## Boundary value analysis

Boundary value analysis is a testing technique that focuses on the values at boundaries; that is, the values near the limits accepted by the system. Using this technique helps to reduce the number of test cases required for adequate testing.

An example of using the boundary value analysis is when you have a name field or address field that accepts a string of characters. The number of characters in the string is a partition; for example, the characters between 1 and 30 is a valid partition with valid boundaries of 1 and 30. The invalid boundaries would be 0 characters (null, just hit the Return key) and 31 characters (too many characters). Both should produce an error message.

The techniques of boundary value analysis are often applied to specific situations or inputs. If different combinations of inputs result in different actions being taken, this can be more difficult to show using equivalence partitioning and boundary value analysis. A decision table is a better technique to use in this situation because it can deal with a combination of inputs and is sometimes referred to as a cause-effect table.

In a web application, this type of testing is used to make sure the data received from the client side is within range.

## Decision tables

A decision table is a way of putting relationships of cause and effect in a logical format to facilitate testing.

[Skip to main content](#)

- If a condition is true, then action 1.
- If a condition is false, then action 2.

Here the tester has identified two outputs (actions 1 and 2) for two conditions (true and false). A decision table can then be drawn up to test these outputs for the different conditions.

An example of a decision table is a credit card company attracting customers by using a discount scheme. The rules of their discount scheme are:

- If a new customer is opening a credit card account, they will get a 15% discount.
- If an existing customer holds a loyalty card, they will get a 10% discount.
- If a customer has a coupon, they will get 20% off. This will stack on top of the loyalty card discount. However, it can't be used with the new-customer discount.

The table shows the decision tree that has been created based on this example. Can you see how true and false conditions have been applied to facilitate calculating the discount?

| Conditions         | Rule 1 | Rule 2 | Rule 3 | Rule 4 | Rule 5 | Rule 6 | Rule 7 | Rule 8 |
|--------------------|--------|--------|--------|--------|--------|--------|--------|--------|
| New customer (15%) | T      | T      | T      | T      | F      | F      | F      | F      |
| Loyalty card (10%) | T      | T      | F      | F      | T      | T      | F      | F      |
| Coupons (20%)      | T      | F      | T      | F      | T      | F      | T      | F      |
| <b>Actions</b>     |        |        |        |        |        |        |        |        |
| Discounts (%)      | X      | X      | 20     | 15     | 30     | 10     | 20     | 0      |

[Skip to main content](#)

## State transition testing

The state transition testing technique is useful for simple AUTs. It's used to test different states of the application where different events or input conditions trigger changes in state. This technique provides a pictorial or tabular representation of the application's behaviour. There are four components to the state transition model:

- **states** of the software
- **transitions** from one state to another
- **events** that initiate a transition
- **actions** that result from a transition.

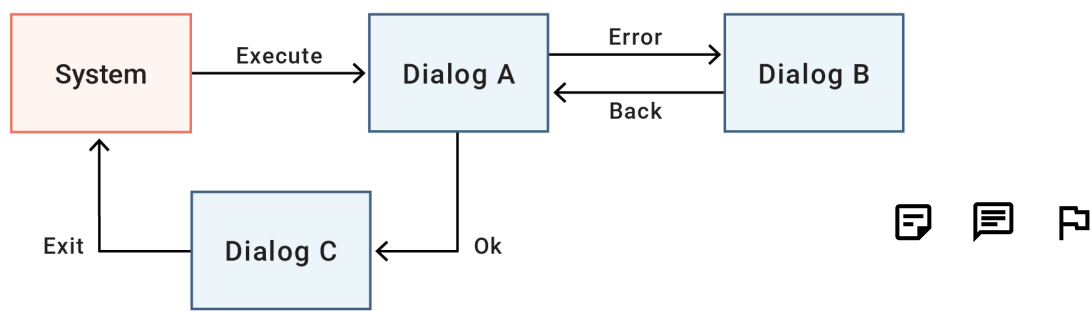
There are two main ways to represent state transition models when testing.

- **State transition diagrams**, where the states of the software are shown in boxed text and the transitions are represented by arrows, as shown below.



[Skip to main content](#)

### State transition diagram



► **State transition tables**, where the states are listed in the left column and the events are described in the top row, as shown below.

| State                | Correct password | Incorrect password |
|----------------------|------------------|--------------------|
| S1 Start             | S5               | S2                 |
| S2 1st try           | S5               | S3                 |
| S3 2nd try           | S5               | S4                 |
| S4 3rd try           | S5               | S6                 |
| S5 Access            | –                | –                  |
| S6 Close application | –                | –                  |

Thoughts on this page?



Skip to main content

